

Thesis Talk Outline

Evaluating Language Models for Coding, Mathematics, and Reasoning

Core Narrative

- Main message: evaluations are not just leaderboards. They are lenses for understanding model behavior, and the lenses we choose shape what the field builds next.
- Start from the post-GPT-4 moment: models looked broadly capable, familiar static benchmarks were saturating, and the strongest systems were increasingly closed.
- Explain why evaluation had to become more diagnostic: a useful benchmark should reveal meaningful shortcomings, explain model behavior, and suggest what to build next.
- Thesis arc: choose faithful metrics, analyze the behavior behind the scores, then use those insights to guide future AI software engineering systems.

Chapter 1: Introduction

- Open with the historical shift: GPT-3 and Codex were impressive but brittle; ChatGPT and GPT-4 changed the community's expectations.
- Emphasize that the surprising part was not only better scores, but broader apparent capability: coding games, proofs, reasoning demos, and general assistant behavior.
- Introduce the central question: what should evaluation do when frontier systems improve quickly and are difficult to understand from the outside?
- Preview the three lessons:
 - Lesson 1: focusing on the right metric without bias is crucial.
 - Lesson 2: looking beyond the score reveals hidden nuances.
 - Lesson 3: what we measure shapes what we build.
- Transition: Chapters 2–4 each develop one lesson through coding, math, and reasoning case studies.

Chapter 2: Right Metrics Without Bias

Chapter setup.

- Frame as the “what should we measure?” chapter.
- Split the chapter into two kinds of work: new metrics for missing capabilities, and controls for confounds that distort existing metrics.

CRUXEval: introduce the benchmark.

- Motivation: code generation benchmarks such as HumanEval and MBPP tested writing code from specs, but programming also requires reading code, understanding it, and simulating execution.
- Introduce CRUXEval as a benchmark for Code Reasoning, Understanding, and eExecution Evaluation.
- Explain the two tasks:
 - Output prediction: given a short Python function and input, predict the output.
 - Input prediction: given a short Python function and desired output, find an input that produces it.
- Talking point: the examples are intentionally simple for humans, because the question is whether models have a basic mental model of execution at all.

CRUXEval: benchmark construction.

- Generate candidate functions and inputs with Code Llama 34B using Python standard-library functions as seeds.
- Execute functions to obtain deterministic input-output pairs.

- Filter aggressively: short programs, no syntax errors, bounded runtime, low arithmetic burden, deterministic behavior, no imports or side effects.
- Select 800 examples after checking that the dataset was large enough to preserve important model comparisons under bootstrap sampling.
- Speaking emphasis: the generate-and-filter recipe made it possible to find simple examples that frontier models still failed.

CRUXEval: evaluation.

- Evaluate open and closed coding models, including StarCoder, Mistral, WizardCoder, Phi, Phind, Code Llama, DeepSeek Coder, GPT-3.5, and GPT-4.
- Use execution-based correctness for both tasks: generated output must match actual output; generated input must satisfy the assertion.
- Report pass@1 and pass@5 with bootstrap intervals and paired comparisons.
- Key result: GPT-4 was strongest, but still failed many simple examples; the best open models failed over half the time.
- Modern framing: today's models largely saturate the original benchmark, which reinforces that the benchmark named a capability at the right moment.

CRUXEval: impact of anonymizing functions.

- Motivation: rule out the concern that models were exploiting memorized or semantically suggestive variable names.
- Ablation: replace variable names with anonymous identifiers such as `x1`, `x2`, ... while preserving function semantics.
- Evaluation: test Code Llama 7B, 13B, and 34B with the same prompts on the anonymized subset.
- Takeaway: relative performance stayed similar, strengthening the case that the benchmark measured execution reasoning rather than memorization or name cues.

SlopCodeBench.

- Introduce the motivation: as models moved into longer software workflows, correctness alone no longer captured whether generated code remained maintainable.
- Benchmark setup: study iterative coding trajectories rather than one-shot function generation.
- Metrics: verbosity and structural erosion as proxies for code slop.
- Evaluation: track correctness, cost, and quality over time, then calibrate against human repository histories.
- Takeaway: models can keep making apparent task progress while degrading code structure, which matters for real software engineering.

IneqMath.

- Introduce the task: solving inequality proof problems.
- Dataset/metric setup: compare final-answer accuracy with step-wise soundness of the reasoning trace.
- Key result: a model can frequently land on the correct final answer while producing invalid proof steps.
- Takeaway: grading only the final answer can support the wrong conclusion about mathematical reasoning ability.

Counterfeit Conundrum.

- Introduce the problem: evaluating whether code models understand incorrect code.
- Benchmark idea: create or identify “counterfeit” programs that are wrong but plausible, avoiding trivial negative examples.
- Evaluation target: correctness checking, execution, and program repair can all look easier when negatives are too obvious.
- Takeaway: negative sample construction is a confound; filtering trivial incorrect samples changes how we interpret model accuracy.

LiveCodeBench.

- Introduce the problem: static public coding benchmarks are vulnerable to contamination and benchmark overfitting.
- Benchmark construction: collect time-indexed coding tasks and evaluate models according to problem release dates.
- Evaluation: compare performance before and after model data cutoffs; include a broader suite of code-related tasks.
- Takeaway: accounting for time and contamination can substantially change model comparisons.

Chapter transition.

- Chapter 2 changes the question from “what score did the model get?” to “what did we measure, and what biases might distort it?”
- Chapter 3 asks what model behavior produced the score.

Chapter 3: Beyond the Score

Chapter setup.

- Frame as the diagnosis chapter. Aggregate scores rank systems, but deeper analysis reveals which capabilities transfer, which failures are distinct, and where future methods should intervene.

CRUXEval quantitative analysis.

- Compare CRUXEval against HumanEval-style code generation.
- Key observation: distilled models improved strongly on HumanEval but not similarly on CRUXEval.
- Interpretation: fine-tuning on natural-language-to-code data can improve code generation without improving underlying execution reasoning.
- Mention the correlation between input and output prediction, and contrast it with the weaker relationship to HumanEval for distilled models.

CRUXEval prompting and fine-tuning analysis.

- Chain-of-thought: helped GPT-4 more than other models; helped output prediction more reliably than input prediction.
- Fine-tuning: input-output assertion data improved scores, but did not make the task solved.
- Qualitative failures: even GPT-4 made mistakes on short programs, sometimes due to brittle reasoning over variables, order, or execution state.
- Takeaway: the score exposed a gap, but the analyses explained what kinds of code reasoning remained hard.

Counterfeit Conundrum analysis.

- Ask what kinds of incorrect programs confuse models.
- Analyze whether counterfeits depend on problem difficulty or generator strength.
- Takeaway: subtle incorrectness is not merely a weak-model or hard-problem artifact; it is a general phenomenon in code understanding.

LiveCodeBench analysis.

- Use the benchmark to examine HumanEval overfitting, post-training effects, and open versus closed model gaps.
- Speaking emphasis: high scores on familiar benchmarks can be ambiguous without time-controlled analysis.
- Takeaway: deeper analysis can distinguish broad capability from adaptation to specific benchmark distributions.

LINC.

- Introduce the task: natural-language logical reasoning over premises and conclusions.
- Method: translate natural language into first-order logic, then run a symbolic prover.
- Evaluation: compare LINC against chain-of-thought across logical reasoning benchmarks.
- Surface result: LINC often outperformed chain-of-thought.
- Deeper result: LINC and chain-of-thought had distinct failure modes; LINC could fail at formalization while chain-of-thought failed at natural-language deduction.

- Takeaway: method comparisons should examine failure structure, not just final accuracy.

Chapter transition.

- Chapter 3 turns benchmark results into scientific understanding.
- Chapter 4 asks what happens when these capabilities and failure modes become shared targets for the community.

Chapter 4: What We Measure Shapes What We Build

Chapter setup.

- Frame as the impact chapter. A benchmark need not remain difficult forever; its lasting value can be naming a capability and making it actionable.

CRUXEval impact.

- Introduce the claim: CRUXEval made execution reasoning visible as a capability beyond generation.
- Benchmark follow-ups: extensions to more languages, runtime state prediction, semantic reasoning, and other code-reasoning tasks.
- Training follow-ups: works used execution traces, input-output prediction, and trace rationales as supervision.
- Model reporting: CRUXEval became a diagnostic benchmark in later code model releases.
- Takeaway: once execution reasoning was measurable, it became something model builders could train for and report on.

LiveCodeBench impact.

- Introduce the methodological contribution: time-indexed, contamination-aware evaluation.
- Explain why it mattered: coding benchmarks need maintenance, not just one fixed release.
- Takeaway: LiveCodeBench changed how model comparisons should be interpreted in a fast-moving field.

VibeCodeBench.

- Introduce the motivation: newer models are used for full applications, not only isolated functions or patches.
- Benchmark design: evaluate end-to-end web application development with flexible outputs.
- Evaluation: combine functional requirements, visual/UI criteria, and human-aligned judgments.
- Takeaway: broader tasks require evaluation methods that can handle product-level outcomes and many valid implementations.

LeanDojo.

- Introduce the setting: theorem proving with language models in Lean.
- Benchmark/toolkit construction: extract data from Lean, expose premise context, and allow interaction with the Lean environment.
- ReProver: retrieval-augmented tactic generation via premise selection and context augmentation.
- Impact: LeanDojo became infrastructure that enabled future theorem-proving research.
- Takeaway: an evaluation can shape a field by becoming the environment researchers build on.

ProofOptimizer.

- Introduce the problem: proofs should be correct, but also shorter and more maintainable.
- Method: use the Lean compiler as a verifier while models propose proof transformations.
- Training/evaluation: proof length becomes a measurable optimization target.
- Takeaway: changing the metric changes what systems learn to improve.

Chapter transition.

- Synthesize the three lessons: choose faithful metrics, analyze behavior behind scores, and recognize that evaluations shape future systems.
- Chapter 5 uses these lessons to articulate what AI software engineering should measure and build next.

Chapter 5: Future Research and Remaining Challenges

Chapter setup.

- Frame Chapter 5 as a synthesis rather than a separate survey.
- The chapter asks what the earlier evaluations imply about the future of AI for software engineering.

Tasks in AI software engineering.

- Introduce the taxonomy as a guardrail against overgeneralizing from one benchmark.
- Measures: scope, logical complexity, and level of human intervention.
- Example: CRUXEval isolates execution reasoning at function-level scope, but says little about project-level codebase reasoning.
- Use the taxonomy to broaden the space beyond code generation: transformation, testing, analysis, maintenance, scaffolding, and formal verification.

Challenges.

- Evaluation and benchmarks: continuing Lesson 1, future benchmarks need task diversity, contamination controls, and construct validity.
- Effective tool usage: LeanDojo, ProofOptimizer, SlopCodeBench, and VibeCodeBench all point toward systems that must use feedback, tests, compilers, and tools.
- Long-horizon code planning: SlopCodeBench and ProofOptimizer show that locally correct artifacts can still be poor, verbose, or hard to maintain.
- Semantic understanding: CRUXEval and Counterfeit reveal function-level weaknesses in subtle code understanding; future systems need this at codebase scale.
- High logical complexity and OOD domains: as simple benchmarks saturate, evaluations should continue moving toward capabilities that remain hard.

Paths forward.

- Data: collect richer code data, including traces, types, call graphs, coverage, and invariants. Connect this naturally to CRUXEval follow-up work on execution traces.
- Training: build executable environments for code RL, echoing the role of Lean feedback in ProofOptimizer but extending it to broader software engineering.
- Adaptation: help models learn from specialized and changing codebases rather than assuming static training data covers every API and convention.
- Inference: use retrieval, code navigation, SWE tools, and neurosymbolic methods to ground model reasoning in executable evidence.
- Synthesis: the next stage of AI for code should be feedback-rich, tool-aware, and evaluated at the level of real software workflows.

Conclusion

- Return to the central thesis: a good evaluation reveals a meaningful capability or failure mode, teaches us why it happens, and helps the field decide what to build next.
- Final recap: metrics determine what we can see; deeper analysis determines what we can understand; benchmarks and tools determine what future systems learn to optimize.
- Closing line: as models keep improving, the most useful evaluations will be those that remain scientifically diagnostic rather than merely difficult.